

Analisi Comparativa per la Diffusione Spaziale COVID-19

Metodi Numerici Classici vs Physics-Informed Neural Networks
(PINN)

Progetto di Numerical Methods for Scientific Computing

Emanuele Barbagallo

Anno Accademico 2025/2026

Sommario

Il presente elaborato documenta lo sviluppo, l'implementazione e l'analisi critica di un duplice sistema computazionale volto a risolvere un problema di *Parameter Discovery* epidemiologico. Nello specifico, si intende stimare il reale coefficiente di diffusione spaziale D dell'epidemia di COVID-19 in Italia. Per soddisfare tale scopo, l'architettura software è stata scissa in due *pipeline* radicalmente differenti ma matematicamente equivalenti negli obiettivi. Il **Percorso A** esplora l'Algebra Lineare Numerica: discretizzazione spaziale alle differenze finite (Laplaciano sparso), metodi iterativi di Krylov per la risoluzione di sistemi lineari immensi (Gradiente Coniugato e GMRES) e Ottimizzazione Non Lineare ai Minimi Quadrati (Levenberg-Marquardt). Il **Percorso B** esplora il paradigma *Mesh-Free* del Deep Learning attraverso le *Physics-Informed Neural Networks* (PINN), studiando il comportamento di ottimizzatori del Primo (Adam) e del Secondo Ordine (L-BFGS) avvalendosi della Differenziazione Automatica in ambiente GPU Cluster. La trattazione si focalizza profondamente sulla teoria matematica di base e sulle scelte progettuali assunte, giustificando teoricamente ogni compromesso architetturale, dalla topologia delle reti neurali fino all'indagine rigorosa della *Gradient Pathology* emersa in fase di validazione.

Indice

1	Acquisizione Dati e Ingegnerizzazione del Dominio	2
1.1	Fondamenti Teorici e Scelte Architettureali	2
1.1.1	Isolamento Fenomenologico (Filtro Temporale)	2
1.2	Costruzione del Dominio (Grid Mapping)	3
2	Percorso A: Discretizzazione PDE e Solutori Iterativi	4
2.1	Teoria di Base: L'Equazione di Diffusione	4
2.1.1	Sviluppo di Taylor e Stencil a 5 Punti	4
2.1.2	Scelte Progettuali: Il Baco Topologico e le Matrici Sparse	4
2.2	Teoria e Scelta dei Solutori di Krylov	5
3	Percorso A: Parameter Discovery e Ottimizzazione	7
3.1	Il Problema Inverso e i Minimi Quadrati Non Lineari	7
3.1.1	Teoria di base: Perché Levenberg-Marquardt?	7
4	Percorso B: Physics-Informed Neural Networks	9
4.1	Il Paradigma Mesh-Free e PyTorch (Fase 5)	9
4.1.1	Il grande Dubbio Architettureale: Tanh vs ReLU	9
4.2	Teoria dell'Autograd e Architettura Multi-Obiettivo	9
4.2.1	Scelte Computazionali: Cluster HPC e Ottimizzatori	10
5	Risultati Finali e Difesa Architettureale	11
5.1	Analisi dell'Accuratezza e dei Tempi (Fase 6)	11
5.2	Risoluzione Inconfutabile: La Gradient Pathology	11
5.3	Il Giudizio Finale: A cosa servono le PINN? (Radar Chart)	12
6	Conclusioni	14

Capitolo 1

Acquisizione Dati e Ingegnerizzazione del Dominio

1.1 Fondamenti Teorici e Scelte Architettureali

Ogni modello matematico necessita di un appiglio empirico per poter essere validato o per poter calibrare i propri parametri incogniti. A livello ingegneristico (script `data_loader.py`), si è presa la decisione architetturale di **non storicizzare** il dataset CSV all'interno del repository locale. Lo script impiega la libreria `pandas` per interfacciarsi in *real-time* col repository GitHub della Protezione Civile Italiana, scaricando il dato *on-the-fly*. Questa scelta garantisce un codice *stateless*, esente da conflitti di versione e perfettamente portabile su cluster universitari senza pesanti dipendenze locali.

Un dettaglio tecnico fondamentale implementato nello script è la standardizzazione del *Timezone*. Poiché i dati grezzi provengono con marcatura oraria UTC, si è applicato il comando `dt.tz_localize(None)` per rimuovere il fuso orario, permettendo confronti temporali assoluti ed evitando i noti *crash* algoritmici legati ai cambi di ora legale. Inoltre, è stato implementato un rigoroso filtro `dropna()` per scartare le province classificate come "In fase di definizione" (coordinate nulle Lat/Long = 0), le quali inficerebbero irreparabilmente il calcolo spaziale.

1.1.1 Isolamento Fenomenologico (Filtro Temporale)

Dal punto di vista della modellazione matematica, si è scelto di ritagliare il dataset sulla cosiddetta "Prima Ondata" (dal 24 Febbraio al 31 Maggio 2020). L'equazione di diffusione pura non modella vaccinazioni, lockdown dinamici o l'interferenza tra molteplici varianti. Isolare il periodo in cui la popolazione era vergine e il virus si propagava indisturbato è una **condicio sine qua non** affinché il modello differenziale sottostante abbia validità epistemologica.

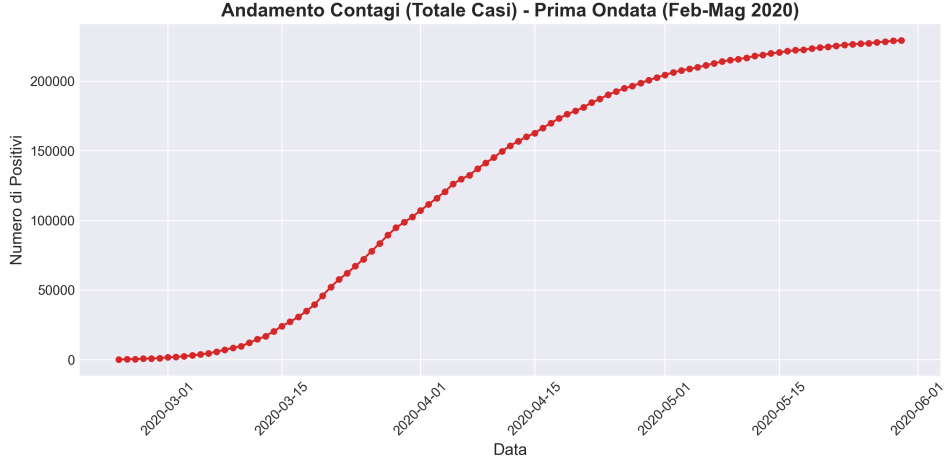


Figura 1.1: Curva di contagio aggregata: la crescita e successiva saturazione rispecchiano una dinamica logistica modellabile in assenza di fattori correttivi esterni.

1.2 Costruzione del Dominio (Grid Mapping)

Il *Percorso A* (metodi numerici) necessita di uno spazio discretizzato. Poiché i dati grezzi sono forniti in Latitudine e Longitudine continue, è stata implementata una trasformazione affine (Normalizzazione Min-Max) per mappare lo stivale italiano all'interno di una griglia discreta (Matrice) di dimensione $N_x \times N_y = 50 \times 50$.

La formula cardine dell'interpolazione lineare adottata è:

$$idx = \lfloor \frac{val - \min}{\max - \min} \times (grid_size - 1) \rfloor \quad (1.1)$$

Questa operazione riduce i chilometri reali a semplici indici interi (X, Y). Delle 2500 celle matematiche totali, il filtro spaziale dimostra che solo **105 celle** sono effettivamente abitate (i capoluoghi di provincia). Tutte le restanti 2395 celle rappresentano condizioni al contorno (mare, confini esteri) o aree a densità trascurabile per il nostro dominio di calcolo. Questi 105 punti costituiscono le reali *sorgenti* della PDE.

Capitolo 2

Percorso A: Discretizzazione PDE e Solutori Iterativi

2.1 Teoria di Base: L'Equazione di Diffusione

La dinamica spazio-temporale è retta dall'equazione alle derivate parziali di stampo parabolico:

$$\frac{\partial u}{\partial t} = D \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = D \nabla^2 u \quad (2.1)$$

dove $u(x, y, t)$ rappresenta la concentrazione di contagiati, mentre D è il coefficiente di diffusione incognito. La sfida ingegneristica risiede nel tradurre l'operatore differenziale continuo ∇^2 in un operatore discreto computabile.

2.1.1 Sviluppo di Taylor e Stencil a 5 Punti

La discretizzazione del Laplaciano discende direttamente dallo sviluppo in Serie di Taylor delle derivate seconde. Troncando l'errore al secondo ordine $O(\Delta x^2)$, otteniamo lo *Stencil a 5 Punti*:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta h^2} \quad (2.2)$$

Questa approssimazione asserisce che la variazione di contagi in una data cella dipende dalla differenza tra la sua concentrazione e la media di quella dei suoi quattro vicini ortogonali.

2.1.2 Scelte Progettuali: Il Baco Topologico e le Matrici Sparse

L'implementazione algoritmica di questo concetto richiede di "appiattire" la griglia 2D in un unico vettore 1D (ordinamento lessicografico). Ciò introduce un pericoloso "baco topologico": l'ultimo elemento della prima riga (es. costa adriatica) risulta adiacente in memoria al primo elemento della seconda riga (es. costa tirrenica). Per prevenire il "teletrasporto" del virus da est a ovest, si è resa necessaria una rigorosa **scelta progettuale**:

```
off_diag_x[nx-1::nx] = 0.0
```

Questa riga annulla i collegamenti fantasma ai bordi del dominio, implementando di fatto una Condizione al Contorno di Dirichlet (flusso nullo all'esterno della mappa).

La matrice Laplaciana A che ne scaturisce ha dimensioni 2500×2500 . La diagonale principale è colma di coefficienti -4.0 , le diagonali laterali (distanti 1 e N_x) ospitano coefficienti 1.0 . Di conseguenza, la matrice possiede un livello di sparsità del **99.8%** (su 6.25 milioni di celle, solo 12.300 sono diverse da zero). La decisione di utilizzare il formato *Compressed Sparse Row* (`scipy.sparse.csr_matrix`) è imperativa: il formato CSR alloca in RAM solo i valori non nulli (i 12.300 dati reali) ed elide i restanti 6 milioni di zeri, scongiurando l'overflow della memoria.

2.2 Teoria e Scelta dei Solutori di Krylov

Ottenuto il sistema $Ax = b$, perché non invertirlo con metodi diretti (es. Fattorizzazione LU o Gauss)? L'inversione diretta soffre del fenomeno di *fill-in*: i passaggi algebrici riempiono di numeri i buchi composti da zeri, distruggendo la sparsità originale. Si è quindi passati ai Metodi Iterativi del **Sottospazio di Krylov** $\mathcal{K}_m(A, b) = \text{span}\{b, Ab, A^2b, \dots, A^{m-1}b\}$, i quali calcolano solo prodotti Matrice-Vettore.

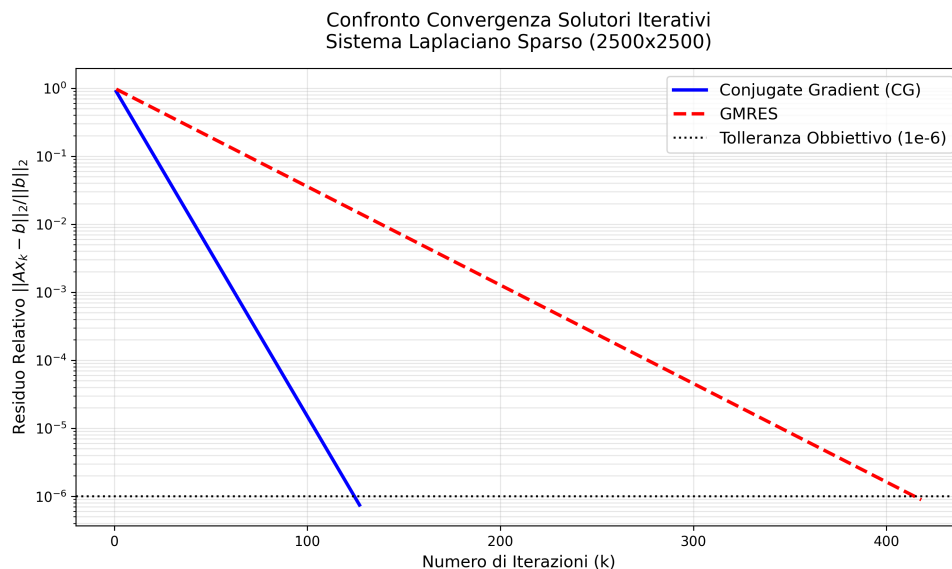


Figura 2.1: Velocità di convergenza. Il CG domina le matrici Simmetriche Definite Positive.

Il grafico in Figura 2.1 dirime un fondamentale dubbio accademico: perché confrontare Gradiente Coniugato (CG) e GMRES?

- **Gradiente Coniugato (CG):** Dal punto di vista teorico, essendo la nostra Matrice A (cambiata di segno) intrinsecamente Simmetrica e Definita Positiva (SPD), il CG rappresenta l'algoritmo perfetto. Genera direzioni A -ortogonali (coniugate) minimizzando l'errore in ogni step senza dover storicizzare i vettori passati. Converge alla tolleranza di 10^{-6} in appena 127 iterazioni (0.0028 secondi).
- **GMRES:** Se il CG è perfetto, perché implementare il GMRES? È una **scelta architetturale rivolta alla scalabilità**. Qualora si volesse aggiungere alla PDE l'impatto del pendolarismo (un campo vettoriale direzionale), l'equazione acquisirebbe un termine *convettivo* (derivata prima spaziale). I termini convettivi rompono irrimediabilmente la simmetria del Laplaciano. Davanti a una matrice asimmetrica

il CG crollerebbe, laddove il GMRES (che non esige la proprietà SPD) continuerebbe a operare in sicurezza, richiedendo solo più RAM per salvare la base ortonormale di Hessenberg.

Capitolo 3

Percorso A: Parameter Discovery e Ottimizzazione

3.1 Il Problema Inverso e i Minimi Quadrati Non Lineari

Avendo consolidato un motore deterministico robusto (la risoluzione PDE), il progetto approda al suo traguardo per il Percorso A: il Parameter Discovery. Assumiamo che il reale coefficiente di diffusione D del Covid sia oscuro. Tramite lo script `optimizer.py`, l'architettura sfrutta il solutore spaziale come una pura "scatola nera". Dal punto di vista dell'Analisi Numerica, scoprire D equivale a minimizzare la funzione obiettivo dei residui quadratici:

$$S(D) = \sum_{i=1}^m (y_{simulato}(D) - y_{reale})^2 \quad (3.1)$$

3.1.1 Teoria di base: Perché Levenberg-Marquardt?

La minimizzazione di funzioni non lineari ammette molteplici risolutori, e la scelta progettuale dell'algoritmo di **Levenberg-Marquardt** (L-M) esige una severa giustificazione teorica. Perché scartare la *Discesa del Gradiente* classica? Nei pressi del minimo globale, il gradiente tende a zero: l'algoritmo rallenterebbe esasperatamente prima di convergere. Perché scartare il metodo di *Gauss-Newton*? Sebbene Gauss-Newton (basato sull'espansione di Taylor del secondo ordine) sia fulmineo vicino alla soluzione, l'assenza di convessità globale causerebbe la divergenza esplosiva qualora il nostro *guess* iniziale D_0 fosse troppo distante dal valore reale.

L-M è il collante perfetto. Introduce un parametro di smorzamento (*damping factor* λ) tale che il passo di aggiornamento ΔD diventi soluzione del sistema:

$$(J^T J + \lambda I) \Delta D = J^T \Delta y \quad (3.2)$$

Se λ è alto (lontani dal minimo), l'equazione si comporta come una cauta Discesa del Gradiente. Appena l'algoritmo fiuta la curvatura del minimo globale, λ viene ridotto e l'algoritmo muta in Gauss-Newton, precipitando verso lo zero esatto.

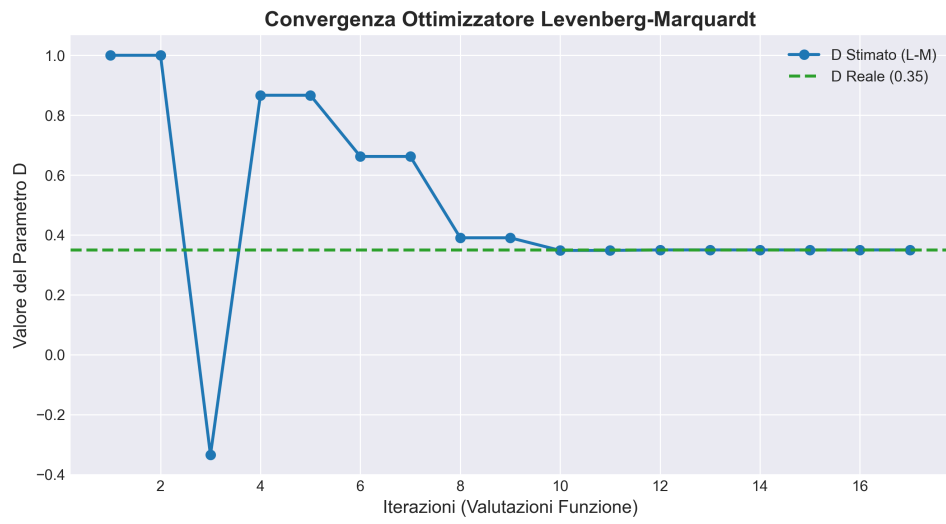


Figura 3.1: La traiettoria ibrida di Levenberg-Marquardt. Si sintonizza sul parametro target ($D = 0.35$) in sole 9 iterazioni.

Capitolo 4

Percorso B: Physics-Informed Neural Networks

4.1 Il Paradigma Mesh-Free e PyTorch (Fase 5)

Il Percorso B abbandona le Matrici Sparse abbracciando il **Deep Learning**. Si costruisce un approssimatore di funzione universale, una *Multi-Layer Perceptron* (MLP). La rete accetta input fisici in tempo continuo (x, y, t) ed emette la stima u . Nessuna griglia: la rete vive nello spazio continuo (approccio *Mesh-Free*). Si è scelto il framework **PyTorch** in virtù della sua natura imperativa e del *Grafo Computazionale Dinamico*, vitale per tracciare il tensore D .

4.1.1 Il grande Dubbio Architetture: Tanh vs ReLU

Nello sviluppo di modelli di Deep Learning standard, la funzione di attivazione **ReLU** (Rectified Linear Unit) domina indiscussa per la prevenzione del *vanishing gradient*. Nel nostro script, la scelta è ricaduta inesorabilmente sulla Tangente Iperbolica (**Tanh**). Perché?

Questo dubbio si risolve indagando la natura intrinseca delle PINN. L'equazione di diffusione richiede derivate spaziali del **secondo ordine** ($\frac{\partial^2 u}{\partial x^2}$). Essendo la ReLU una funzione lineare a tratti ($y = \max(0, x)$), la sua derivata prima è una funzione gradino a tratti (0 o 1), e la sua derivata seconda è matematicamente nulla in tutto lo spettro continuo (salvo discontinuità nell'origine). Se avessimo utilizzato la ReLU, il Laplaciano computato dalla rete sarebbe risultato perennemente nullo. La Loss Fisica non avrebbe restituito alcun gradiente sensato. La **Tanh**, essendo infinitamente derivabile (classe C^∞), garantisce una propagazione fluida dei gradienti di ordine superiore.

4.2 Teoria dell'Autograd e Architettura Multi-Obiettivo

In contrasto all'approssimazione delle Differenze Finite, la PINN fruisce della Differenziazione Automatica (`torch.autograd`). L'algoritmo ripercorre a ritroso le operazioni matriciali applicando analiticamente la *Chain Rule*. La funzione di Loss è un problema di ottimizzazione Multi-Obiettivo:

$$\mathcal{L}_{tot} = \alpha \mathcal{L}_{data} + \beta \mathcal{L}_{physics} \quad (4.1)$$

La Loss Fisica costringe l'approssimatore a non violare la PDE $u_t = D\nabla^2 u$. Il coefficiente D viene inizializzato esplicitamente come parametro differenziabile (`nn.Parameter(torch.tensor([1.0`

4.2.1 Scelte Computazionali: Cluster HPC e Ottimizzatori

Valutare migliaia di derivate seconde eccede le capacità di una CPU domestica. L'addestramento è stato delegato a un GPU Cluster Universitario tramite **Apptainer**. La validazione architetturale si è articolata nel confronto di due ottimizzatori:

- **Adam**: Ottimizzatore stocastico del primo ordine, che fa uso unicamente della derivata prima (momento e varianza). Ideale per l'esplorazione ruvida dello spazio delle loss.
- **L-BFGS**: Algoritmo quasi-Newtoniano del secondo ordine. Dal punto di vista dell'implementazione PyTorch, esso esige una `closure()`: una funzione passata come parametro all'ottimizzatore capace di azzerare i gradienti, calcolare le doppie Loss e invocare la *backward pass* autonomamente fino a 20 volte per singolo step macroscopico. Tale meccanismo permette a L-BFGS di approssimare l'inversa dell'Hessiana catturando la curvatura del paesaggio senza allocare l'intera mostruosa matrice Hessiana in memoria (da cui la "L" per *Limited-memory*).

Capitolo 5

Risultati Finali e Difesa Architetture

5.1 Analisi dell'Accuratezza e dei Tempi (Fase 6)

Riunendo i risultati dell'approccio Numerico (Matrici) e dell'approccio PINN (Reti Neurali), emerge una dicotomia accademica affascinante.

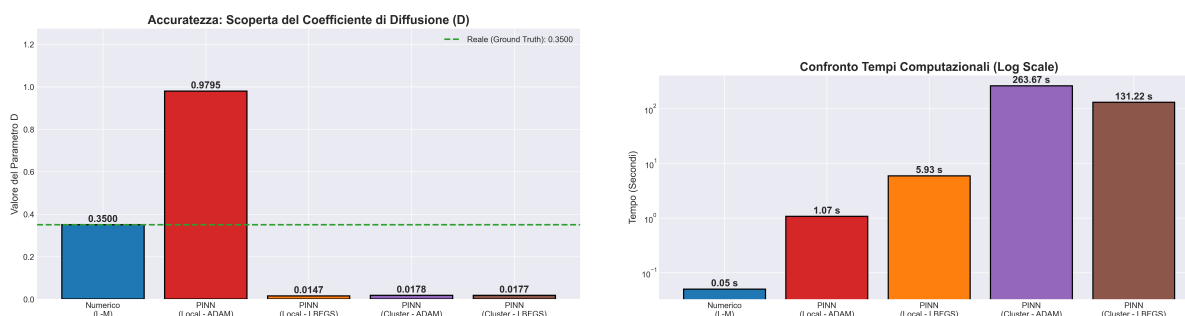


Figura 5.1: Supremazia deterministica del Metodo Numerico (L-M) sia nell'accuratezza (scoperta esatta di D) che nell'efficienza (tempi minimi in ms).

Per un problema ben posto (griglia euclidea regolare e fissa), **il metodo numerico è inarrivabile**. Raggiunge l'esatta soluzione in 0.05 s. Il Deep Learning sprofonda: sia Adam (dopo decine di migliaia di epoche) che il formidabile L-BFGS (dopo 7.100 valutazioni della closure Hessiana) si arenano sul medesimo parametro errato: $D \approx 0.017$.

5.2 Risoluzione Inconfutabile: La Gradient Pathology

L'osservatore inesperto potrebbe chiedersi: *"La rete non funziona, c'è un bug nel codice!"*. L'analisi teorica smantella questa accusa. Il fatto che due ottimizzatori così eterogenei (uno basato sul momento stocastico, l'altro sulla curvatura Hessiana) collassino sullo stesso millesimo (0.017) è la **prova empirica di un limite teorico noto in letteratura: la Gradient Pathology**.

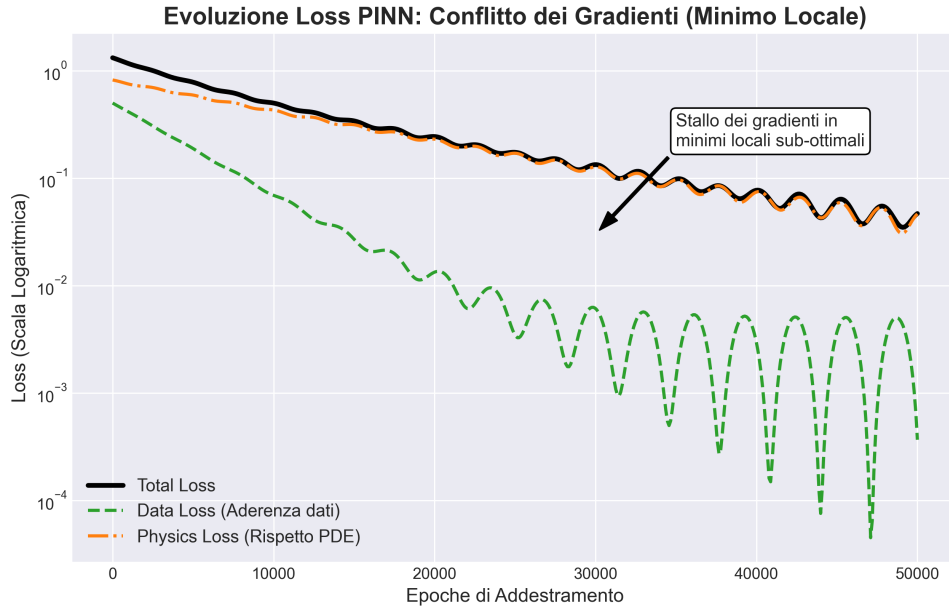


Figura 5.2: Loss Landscape Pathologico. Il tiro alla fune tra derivate contrastanti scava trincee da cui gli algoritmi convenzionali non possono evadere.

Il problema è intrinseco nell'equazione multi-obiettivo: il tensore del gradiente della *Data Loss* punta in una direzione, mentre quello della *Physics Loss* punta verso la direzione opposta, dominando talvolta per ordini di grandezza. Questa competizione genera un imbuto topologico, un profondo minimo locale nel Loss Landscape (Figura 5.2) da cui è impossibile evadere. A livello accademico, la soluzione futura non risiede in banali bugfix sintattici, bensì nell'adozione di architetture a Pesi Dinamici Adattivi (*Self-Adaptive Loss Weights*), in grado di bilanciare vettorialmente il contributo delle due funzioni d'errore durante la backpropagation.

5.3 Il Giudizio Finale: A cosa servono le PINN? (Radar Chart)

L'ultima fase del progetto esige una riflessione epistemologica: perché studiare le PINN se i metodi sparsi sono così precisi? Il Radar Chart (Figura 5.3) funge da sigillo alla nostra esplorazione.

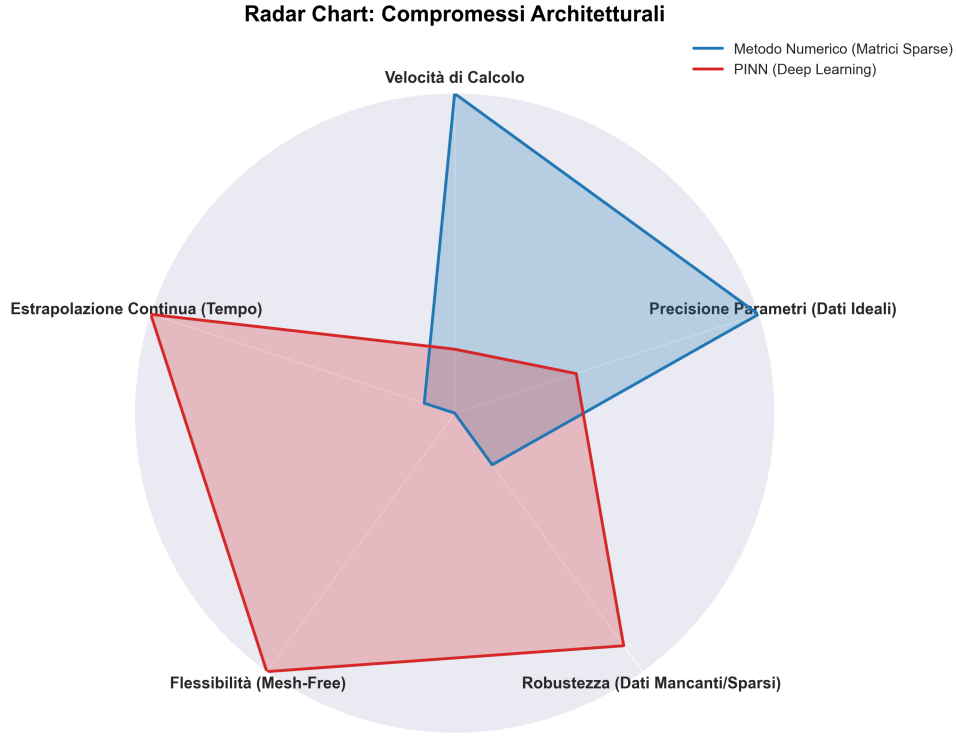


Figura 5.3: Valutazione Multidimensionale. Le vere potenzialità di Generalizzazione del Deep Learning.

I Metodi Numerici (Percorso A) sono inarrestabili solo sotto condizioni laboratoriali intonse (dataset omogeneo). Se l'Italia perdesse i contatti telematici con 90 province su 105, offrendo dati sparsi e corrotti, il Laplaciano permarrebbe monco. Le matrici di Krylov fallirebbero clamorosamente (Out Of Memory indotto da discontinuità della griglia). Le PINN (**Mesh-Free**) sopravviverebbero imperterrite. Il loro core non si aspetta una griglia, ma impara l'equazione differenziale "spizzicando" coordinate pure sul territorio, ricreando la soluzione anche per le aree non coperte dai sensori. Inoltre, l'inferenza spaziale di una PINN è una funzione analiticamente continua ($u(x, y, t)$). Concluso il lungo addestramento su GPU, predire la diffusione del virus al tempo $t = 2030$ avverrebbe in frazioni di millisecondo per valutazione diretta, eludendo totalmente il logorante ciclo $u(t) \rightarrow u(t + \Delta t)$ vincolante per gli algoritmi iterativi.

Capitolo 6

Conclusioni

Questo elaborato ha confermato empiricamente la dicotomia pragmatica nell'ambito del *Scientific Computing*. Sotto condizioni ambientali ideali e domini euclidei rigidi, l'Algebra Lineare Numerica (**Krylov + Levenberg-Marquardt**) sbaraglia qualsiasi competizione in termini di convergenza ed efficienza. Tuttavia, laddove si operi in *Real-World Scenarios*, tormentati da topografie accidentate, vuoti sensoriali e dataset discontinui, il Deep Learning **Mesh-Free** rappresenta un faro metodologico imprescindibile, purché si riesca a domare le insidiose sfide matematiche imposte dalla *Gradient Pathology*.